



CS1004 - Object Oriented Programming

LAB # 3

DYNAMIC MEMORY ALLOCATION

Lab Objectives

- Master advanced pointer concepts including const, null, void, dangling, and wild pointers
- Understand pointer arithmetic and its application with arrays
- Learn dynamic memory allocation using new and delete keywords
- Implement multi-dimensional arrays using various pointer techniques
- Work with character pointers and string manipulation
- Understand the difference between stack and heap memory
- Practice function pointers and pointer to pointer concepts

CONCEPT REVIEW

1. Pointers in Functions

Pointers can be passed to and returned from functions, enabling functions to modify variables outside their scope and work with dynamic memory.

Example:

```
void modifyValue(int* ptr) {  
    *ptr = 100; // Modifies the original variable  
}
```

2. Const Variable (int)

A constant variable cannot be modified after initialization. It must be initialized when declared.

Example:

```
const int MAX_SIZE = 100; // Cannot be changed  
// MAX_SIZE = 200; // ERROR: Cannot modify const
```

3. Null Pointer

A null pointer is a pointer that doesn't point to any memory location. It's initialized with NULL, nullptr (C++11), or 0. Always check if a pointer is null before dereferencing to avoid crashes.

Example:

```
int* ptr = nullptr; // Modern C++ way
if (ptr != nullptr) { // Always check before using
    cout << *ptr;
}
```

4. Void Pointer

A void pointer (void*) is a generic pointer that can point to any data type. It must be type-cast before dereferencing.

Example:

```
void* ptr;
int x = 10;
ptr = &x;
cout << *(int*)ptr; // Must cast to int* before dereferencing
```

5. Dangling Pointer

A dangling pointer points to memory that has been freed or deallocated. Using it causes undefined behavior. Always set pointers to nullptr after deleting.

Example:

```
int* ptr = new int(42);
delete ptr; // Memory is freed
ptr = nullptr; // Good practice: Avoid dangling pointer
// *ptr; // DANGEROUS: Would be undefined behavior without nullptr
```

6. Wild Pointer

A wild pointer is an uninitialized pointer that contains garbage value. Always initialize pointers when declaring them.

Example:

```
int* ptr; // Wild pointer - contains garbage
// *ptr = 10; // DANGEROUS: Undefined behavior
int* safe_ptr = nullptr; // GOOD: Initialized
```

7. Casting Pointers

Pointer casting converts a pointer from one type to another. Use static_cast, dynamic_cast, const_cast, or reinterpret_cast in C++.

Example:

```
double d = 3.14;
void* vptr = &d;
double* dptr = static_cast<double*>(vptr);
```

8. Constant Pointer (int)

A constant pointer cannot change the address it points to, but can modify the value at that address.

Syntax: `type* const ptr`

Example:

```
int x = 10, y = 20;  
int* const ptr = &x; // Constant pointer  
*ptr = 15; // OK: Can modify value  
// ptr = &y; // ERROR: Cannot change address
```

9. Pointer to Constant (int)

A pointer to constant cannot modify the value it points to, but can point to different addresses.

Syntax: `const type* ptr` or `type const* ptr`

Example:

```
int x = 10, y = 20;  
const int* ptr = &x; // Pointer to constant  
ptr = &y; // OK: Can change address  
*ptr = 15; // ERROR: Cannot modify value
```

10. Constant Pointer to Constant (int)

Cannot change the address or modify the value.

Syntax: `const type* const ptr`

Example:

```
int x = 10, y = 20;  
const int* const ptr = &x; // Constant pointer to constant  
// *ptr = 15; // ERROR: Cannot modify value  
// ptr = &y; // ERROR: Cannot change address
```

11. Pointer to Pointer (int)

A pointer that stores the address of another pointer. Used for dynamic 2D arrays and modifying pointers in functions.

Example:

```
int x = 42;  
int* ptr = &x; // Pointer to int  
int** pptr = &ptr; // Pointer to pointer  
cout << **pptr; // Prints 42
```

12. Function Pointer

A pointer that stores the address of a function. Enables callback functions and dynamic function calls.

Syntax: `return_type (*pointer_name)(parameter_types)`

Example:

```
int add(int a, int b) { return a + b; }  
int (*funcPtr)(int, int) = add;  
cout << funcPtr(5, 3); // Prints 8
```

13. Pointer Arithmetic

Pointers can be incremented, decremented, added, or subtracted. The arithmetic is scaled by the size of the data type.

Example:

```
int arr[] = {10, 20, 30, 40, 50};  
int* ptr = arr;  
cout << *ptr; // 10  
cout << *(ptr+2); // 30 (moves 2 integers forward)  
ptr++; // Points to arr[1]
```

14. Pointers and Arrays (int)

Array name acts as a constant pointer to the first element. Pointers can traverse arrays.

Example:

```
int arr[] = {1, 2, 3, 4, 5};  
int* ptr = arr; // arr decays to pointer  
// arr[i] is same as *(arr + i) or *(ptr + i)
```

15. Array of Pointers (int)

An array where each element is a pointer. Useful for arrays of strings or jagged arrays.

Example:

```
int a = 10, b = 20, c = 30;  
int* arr[3] = {&a, &b, &c}; // Array of 3 int pointers  
cout << *arr[1]; // Prints 20
```

16. Pointer to an Array (1D and 2D) (int)

A pointer that points to an entire array. Different from array of pointers.

Syntax: type (*ptr)[size]

1D Array:

```
int arr[5] = {1, 2, 3, 4, 5};  
int (*ptr)[5] = &arr; // Pointer to array of 5 ints
```

2D Array:

```
int arr[3][4] = {{1,2,3,4}, {5,6,7,8}, {9,10,11,12}};  
int (*ptr)[4] = arr; // Pointer to array of 4 ints
```



17. Stack vs Heap

Stack: Automatic memory, fast allocation, limited size, automatically deallocated. Used for local variables.

Heap: Dynamic memory, slower allocation, large size, manual deallocation required using delete. Used for dynamic arrays and objects.

Example:

```
int x = 10;           // Stack
int* ptr = new int;    // Heap - must delete later
```

18. Usage of New and Delete Keyword (int)

new: Allocates memory on heap and returns pointer.

delete: Deallocates memory allocated by new.

Example:

```
int* ptr = new int(42); // Allocate and initialize
cout << *ptr;
delete ptr; // Free memory
ptr = nullptr; // Good practice
```

19. Pointers and 1D Array (Dynamic Memory Allocation)

Dynamically allocate arrays whose size is determined at runtime.

Example:

```
int size;
cin >> size;
int* arr = new int[size]; // Dynamic allocation
// Use array
delete[] arr; // Must use delete[] for arrays
arr = nullptr;
```

20. Multi-Dimensional Arrays (int)

Arrays with multiple dimensions. Can be accessed using pointer arithmetic.

Static 2D Array:

```
int arr[3][4]; // 3 rows, 4 columns
arr[1][2] = 10;
```

Dynamic 2D Array:

```
int** arr = new int*[rows];
for(int i = 0; i < rows; i++)
    arr[i] = new int[cols];
```

21. Accessing Multi-Dimensional Array with Single Pointer

2D array can be treated as 1D array in memory using pointer arithmetic.

Formula: $\text{arr}[i][j] = *(\text{ptr} + i * \text{cols} + j)$

Example:

```
int rows = 3, cols = 4;  
int* arr = new int[rows * cols];  
arr[i*cols + j] = value; // Access element [i][j]
```

22. Accessing Multi-Dimensional Array Using Pointer to Array

Use pointer to array for better type safety.

Example:

```
int (*ptr)[4]; // Pointer to array of 4 ints  
ptr = new int[3][4];  
ptr[i][j] = value; // Access like regular 2D array
```

23. Accessing Multi-Dimensional Array Using Double Pointer (DMA)

Most flexible approach for dynamic 2D arrays with variable row sizes.

Example:

```
int rows = 3, cols = 4;  
int** arr = new int*[rows];  
for(int i = 0; i < rows; i++)  
    arr[i] = new int[cols];  
// Use: arr[i][j]  
// Deallocate:  
for(int i = 0; i < rows; i++)  
    delete[] arr[i];  
delete[] arr;
```

24. Using Triple Pointers for 3D Arrays (DMA)

Extend double pointer concept to three dimensions.

Example:

```
int x = 2, y = 3, z = 4;  
int*** arr = new int**[x];  
for(int i = 0; i < x; i++) {  
    arr[i] = new int*[y];  
    for(int j = 0; j < y; j++)  
        arr[i][j] = new int[z];  
}
```

25. Single Character Pointer (char*)

Points to a single character or a C-string (null-terminated character array).

Example:

```
char ch = 'A';  
char* ptr = &ch;  
cout << *ptr; // Prints 'A'
```

26. String as a Pointer (char* str)



C-style strings are character arrays terminated with null character ('\0'). Can be manipulated using character pointers.

Example:

```
char* str = "Hello"; // Points to string literal (read-only)
cout << str; // Prints "Hello"
cout << str[1]; // Prints 'e'
```

27. Character Array vs. Character Pointer

Character Array:

- Stores actual characters in array
- Modifiable
- Fixed size

Character Pointer:

- Points to string literal (read-only) or dynamically allocated memory
- Can point to different strings
- More flexible

Example:

```
char arr[] = "Hello"; // Array - modifiable
arr[0] = 'h'; // OK

char* ptr = "Hello"; // Pointer to literal - read-only
// ptr[0] = 'h'; // UNDEFINED BEHAVIOR
```

LAB TASKS

Task 1: Memory Address Comparison

Write a program that:

- Creates integer num = 42
- Creates pointer int* ptr = &num
- Creates reference int& ref = num
- Compares addresses of all three

Sample Output:

Address of num: 0x7ffd...

Address stored in ptr: 0x7ffd... (same as above)

Address of ref: 0x7ffd... (same as above)

All point to same memory location.

Task 2: Multiple References

Write a program that:

- Creates integer value = 100

- Creates two references ref1 and ref2 to same variable
- Modifies through one reference, shows all changed

Sample Output:

Original value: 100

ref1: 100, ref2: 100

After modifying through ref1:

value: 200, ref1: 200, ref2: 200

Task 3: Null Pointer Safety

Write a program that:

- Creates function accepting pointer parameter
- Checks for nullptr before dereferencing
- Demonstrates safe and unsafe usage

Sample Output:

Safe call: Value = 50

Unsafe call (nullptr): Error: Null pointer detected!

Task 4: Stack vs Heap Demo

Write a program that:

- Creates local variable (stack)
- Creates dynamically allocated variable (heap)
- Compares their memory addresses

Sample Output:

Stack variable address: 0x7ffd... (higher address)

Heap variable address: 0x... (lower address)

Stack grows downward, heap grows upward.

Task 5: Character Array via Pointer

Write a program that:

- Creates char str[] = "Mutable"
- Creates pointer char* p = str
- Modifies string through pointer
- Shows original array changed

Sample Output:

Original: Mutable

After pointer modification: Changeable

Task 6: Pointer to 2D Array

Write a program that:

- Creates static 2D array int matrix[3][3]
- Uses single pointer to access all elements
- Displays matrix using pointer arithmetic

Sample Output:

Matrix using single pointer:

1 2 3

4 5 6

7 8 9

Task 7: Array of Function Pointers



Write a program that:

- Implements math operations: square, cube, factorial
- Stores functions in array of function pointers
- Calls functions through array

Sample Output:

Square of 5: 25

Cube of 5: 125

Factorial of 5: 120

Task 8: Dynamic String Array

Write a program that:

- Asks for number of strings
- Dynamically allocates array of strings using char**
- Takes input, displays, frees memory

Sample Input:

Number of strings: 3

String 1: Apple

String 2: Banana

String 3: Cherry

Sample Output:

Strings:

1. Apple

2. Banana

3. Cherry

Memory freed.

Task 9: Sparse Matrix using DMA

Write a program that:

- Implements sparse matrix using array of pointers
- Stores only non-zero elements
- Displays original and sparse representations

Sample Output:

text

Original 4x4 matrix:

0 0 3 0

0 0 0 0

0 5 0 0

0 0 0 9

Sparse representation:

(0,2): 3

(2,1): 5

(3,3): 9

Task 10: Callback System

Write a program that:

- Implements callback registration using function pointers
- Has onEvent() function that triggers callbacks
- Demonstrates multiple callbacks

Sample Output:

Event occurred!

Callback 1 executed.



Callback 2 executed.
Callback 3 executed.

HOME TASKS

Home Task 1: Array of Strings - Contact Book

Create a contact book system using array of character pointers (`char* arr[]`).

Requirements:

- Store up to 10 contact names using `char*` array
- Dynamically allocate memory for each name
- Implement add, delete, search, and display functions
- Sort names alphabetically
- Properly manage memory allocation and deallocation

Home Task 2: Deep Copy vs Shallow Copy - Dynamic Array Manager

Demonstrate the difference between shallow and deep copy with a dynamic array class-like structure using pointers.

Requirements:

- Create struct `ArrayManager` with `int*` data and `int` size
- Implement `shallowCopy()` - copies pointer only
- Implement `deepCopy()` - allocates new memory and copies data
- Demonstrate problems with shallow copy (modification affects both)
- Show benefits of deep copy (independent copies)